ARTIFICIAL INTELLIGENCE

CO1
AT2

NAME : S.PRITHISH
REG NO : 192421134

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:
Constraints:
i. D1 cannot work Night shift
ii. D2 must work before D3 (Morning < Afternoon < Night)
iii. Only one doctor per shift
iv. D3 cannot work Morning
v. All doctors must be assigned exactly one shift
Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

SOL:

```
doctors = ["D1", "D2", "D3"]
shifts = ["Morning", "Afternoon", "Night"]
assignment = {}

def is_valid(doctor, shift):
    if doctor == "D1" and shift == "Night":
        print("Constraint failed: D1 cannot work Night")
        return False

    if doctor == "D3" and shift == "Morning":
        print("Constraint failed: D3 cannot work Morning")
        return False

    if shift in assignment.values():
        print("Constraint failed: Only one doctor per shift")
        return False

    if doctor == "D3":
        if shifts.index(assignment["D2"]) >= shifts.index(shift):
            print("Constraint failed: D2 must work before D3")
            return False

    return True
```

```
def backtracking(index):
    if index == len(doctors):
        return True

    doctor = doctors[index]

    for shift in shifts:
        print("\nTrying", doctor, "=", shift)

        if doctor == "D3" and "D2" not in assignment:
            continue

        if is_valid(doctor, shift):
            assignment[doctor] = shift
            print("Assignment accepted:", assignment)

            if backtracking(index + 1):
                return True

            print("Backtracking from", doctor, "=", shift)
            del assignment[doctor]

        else:
            print("Assignment rejected")

    return False

if backtracking(0):
    print("\nFinal Valid Schedule")
    for doctor in doctors:
        print(doctor, ":", assignment[doctor])
else:
    print("No valid schedule found")
```

OUTPUT:

```
========== RESTART: C:/Users/admin/A11.py ==========

Trying D1 = Morning
Assignment accepted: {'D1': 'Morning'}

Trying D2 = Morning
Constraint failed: Only one doctor per shift
Assignment rejected

Trying D2 = Afternoon
Assignment accepted: {'D1': 'Morning', 'D2': 'Afternoon'}

Trying D3 = Morning
Constraint failed: D3 cannot work Morning
Assignment rejected

Trying D3 = Afternoon
Constraint failed: Only one doctor per shift
Assignment rejected

Trying D3 = Night
Assignment accepted: {'D1': 'Morning', 'D2': 'Afternoon', 'D3': 'Night'}

Final Valid Schedule
D1 : Morning
D2 : Afternoon
D3 : Night
```

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.
Constraints:
i. Movement allowed: Up, Down, Left, Right
ii. Cost of each move = 1
iii. Cannot pass through obstacles
iv. Use Manhattan Distance heuristic
Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

SOL:

```
from collections import deque

grid = [
    ['S', '.', '#', '.', '.'],
    ['.', '.', '#', '.', '#'],
    ['.', '.', '.', '.', '#'],
    ['#', '#', '.', '#', '.'],
    ['.', '.', '.', '.', 'G']
]

start = (0, 0)
goal = (4, 4)

moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

```
def manhattan(pos):
    return abs(pos[0] - goal[0]) + abs(pos[1] - goal[1])

def bfs():
    queue = deque([start])
    visited = {start}
    parent = {start: None}
    step = 0

    while queue:
        current = queue.popleft()
        step += 1

        print("\nStep:", step)
        print("Expanded Node:", current)
        print("Manhattan Distance:", manhattan(current))

        if current == goal:
            print("Goal Found!")
            break

        for dr, dc in moves:
            nr = current[0] + dr
            nc = current[1] + dc
            neighbor = (nr, nc)

            if (0 <= nr < 5 and 0 <= nc < 5
                    and grid[nr][nc] != '#'
                    and neighbor not in visited):

                visited.add(neighbor)
                parent[neighbor] = current
                queue.append(neighbor)

        print("Queue Update:", list(queue))

    if goal not in parent:
        print("No path found")
        return

    path = []
    node = goal
```

```
    while node is not None:
        path.append(node)
        node = parent[node]

    path.reverse()

    print("\nOptimal Path:")
    print(" -> ".join(map(str, path)))
    print("Minimum Cost:", len(path) - 1)

    print("\nFinal Grid:")
    for r in range(5):
        for c in range(5):
            if (r, c) in path:
                print("*", end=" ")
            else:
                print(grid[r][c], end=" ")
        print()

bfs()
```

OUTPUT :

```
========== RESTART: C:/Users/admin/A11.py ==========

Trying D1 = Morning
Assignment accepted: {'D1': 'Morning'}

Trying D2 = Morning
Constraint failed: Only one doctor per shift
Assignment rejected

Trying D2 = Afternoon
Assignment accepted: {'D1': 'Morning', 'D2': 'Afternoon'}

Trying D3 = Morning
Constraint failed: D3 cannot work Morning
Assignment rejected

Trying D3 = Afternoon
Constraint failed: Only one doctor per shift
Assignment rejected

Trying D3 = Night
Assignment accepted: {'D1': 'Morning', 'D2': 'Afternoon', 'D3': 'Night'}

Final Valid Schedule
D1 : Morning
D2 : Afternoon
D3 : Night
```

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.
The building is represented as a grid of rooms. The robot starts at position S and must reach a

survivor located at G.
Constraints:
The robot can move up, down, left, right
Some cells are blocked (obstacles) and cannot be traversed
The robot has limited sensing capability (can only observe adjacent cells)
Each movement has a cost of 1, but entering risky zones has an additional cost of 2
The robot must minimize total cost while ensuring safe navigation
The robot must update its knowledge dynamically (online search scenario)
Tasks:
a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:
Intelligent agents
Rationality
Environment type
d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

SOL:

```
import heapq

actual_grid = [
    ['S', '.', 'R', '.', '.'],
    ['#', '.', '#', '.', '#'],
    ['.', '.', '.', 'R', '.'],
    ['.', '#', '.', '#', '.'],
    ['.', '.', '.', '.', 'G']
]

rows = 5
cols = 5
start = (0, 0)
goal = (4, 4)

directions = [
    (-1, 0),
    (1, 0),
    (0, -1),
    (0, 1)
]
```

```
known_grid = [['?' for _ in range(cols)] for _ in range(rows)]
known_grid[start[0]][start[1]] = 'S'

def observe(position):
    r, c = position

    for dr, dc in directions:
        nr = r + dr
        nc = c + dc

        if 0 <= nr < rows and 0 <= nc < cols:
            known_grid[nr][nc] = actual_grid[nr][nc]

def ucs(current):
    queue = [(0, current)]
    costs = {current: 0}
    parent = {current: None}

    while queue:
        cost, position = heapq.heappop(queue)

        if cost != costs[position]:
            continue

        if position == goal:
            path = []
            node = goal

            while node is not None:
                path.append(node)
                node = parent[node]

            return path[::-1], cost

        r, c = position

        for dr, dc in directions:
            nr = r + dr
            nc = c + dc

            if not (0 <= nr < rows and 0 <= nc < cols):
                continue
```

```
            cell = known_grid[nr][nc]

            if cell in ('?', '#'):
                continue

            next_position = (nr, nc)
            move_cost = 1

            if cell == 'R':
                move_cost += 2

            new_cost = cost + move_cost

            if new_cost < costs.get(next_position, float('inf')):
                costs[next_position] = new_cost
                parent[next_position] = position
                heapq.heappush(
                    queue, (new_cost, next_position)
                )

    return None, float('inf')

position = start
total_cost = 0
visited = {start}

print("AUTONOMOUS RESCUE ROBOT")
print("Start:", start)
print("Goal:", goal)

while position != goal:
    observe(position)

    print("\nCurrent Position:", position)
    print("Updated Knowledge Map:")

    for row in known_grid:
        print(" ".join(row))

    path, estimated_cost = ucs(position)

    if path is None:
        print("No known safe route. Exploring nearby cells.")
```

```
    possible_moves = []

    r, c = position

    for dr, dc in directions:
        nr, nc = r + dr, c + dc

        if 0 <= nr < rows and 0 <= nc < cols:
            if known_grid[nr][nc] == '?':
                possible_moves.append((nr, nc))

    if not possible_moves:
        print("No further exploration possible.")
        break

    next_position = possible_moves[0]
else:
    if len(path) < 2:
        break

    next_position = path[1]

nr, nc = next_position

if actual_grid[nr][nc] == '#':
    known_grid[nr][nc] = '#'
    print("Blocked cell discovered:", next_position)
    continue

if actual_grid[nr][nc] == 'R':
    total_cost += 3
    print("Risky zone entered:", next_position)
else:
    total_cost += 1

position = next_position
visited.add(position)

print("Robot moved to:", position)
print("Movement cost:", 3 if actual_grid[nr][nc] == 'R' else 1)
print("Total cost so far:", total_cost)

if position == goal:
    break
```

```
    if len(visited) > rows * cols:
        break

observe(position)

print("\nFINAL RESULT")

if position == goal:
    print("Survivor reached successfully!")
    print("Final Position:", position)
    print("Total Cost:", total_cost)
    print("Visited Cells:", visited)
else:
    print("Survivor not reached. Further exploration is required.")
```

OUTPUT :

```
========== RESTART: C:/Users/admin/AI1.py ==========
AUTONOMOUS RESCUE ROBOT
Start: (0, 0)
Goal: (4, 4)

Current Position: (0, 0)
Updated Knowledge Map:
S . ? ? ?
# ? ? ? ?
? ? ? ? ?
? ? ? ? ?
? ? ? ? ?
No known safe route. Exploring nearby cells.
No further exploration possible.

FINAL RESULT
Survivor not reached. Further exploration is required.
```